

AUDITORIA TÉCNICA DE CÓDIGO-FONTE

Relatório de Auditoria de Segurança

Avaliação Aprofundada de Vulnerabilidades e Arquitetura Multi-Tenant

APLICAÇÃO / ALVO Procard / Arms Web (Sistema de Fidelidade & Vouchers)	DATA DA AVALIAÇÃO 08 de Setembro de 2026
STACK IDENTIFICADA Flutter / Dart (FlutterFlow), GoRouter, HTTP REST Gateway	SERVIÇO BACKEND REST API em codeflowbr.online:8080 (Sem RLS / Sem Gateway WAF)
ESCOPO AUDITADO lib/ (Frontend, Auth, Routing, Endpoints, Custom Actions, Assets)	STATUS GERAL 3 Críticas, 4 Altas, 3 Médias, 2 Baixas, 4 Pontos Fortes

Nota Metodológica — Mapeamento para a Stack Local:

O projeto consiste em uma aplicação Flutter Web gerada via FlutterFlow consumindo APIs REST proprietárias via `api_manager.dart` e `api_calls.dart`. Cada uma das 5 categorias foi mapeada conforme:

- **1. Banco Sem Tranca:** Como o backend não usa Supabase/Firebase, a verificação focou em queries de listagem em endpoints REST (`/customer`, `/dashboard`, `/discount`, `/partner`, `/trade`) que retornam dados consolidados sem filtrar pelo `tenant_id` da sessão.
- **2. Permissão no Navegador:** Mapeamento dos gates condicionais no cliente (ex: checagem de `role == 2` em `login_widget.dart`) cruzados com a ausência de middleware e tokens de autorização nos endpoints administrativos no backend.
- **3. IDOR:** Varredura exaustiva em todos os métodos de exclusão, edição e leitura que aceitam IDs numéricos (`idDiscount`, `idCustomer`, `idPartner`, `idProduto`, etc.) sem checagem de propriedade.
- **4. Chaves Expostas:** Investigação de strings de conexão, credenciais embutidas, persistência desprotegida de JWT em `SharedPreferences` (`localStorage`) e constantes estáticas (`tenantId = '1'`).
- **5. Inputs Sem Tratamento:** Inspeção de injeção de JSON por interpolação de strings em payloads REST, validação de esquemas de URL em `launchURL` e `upload` sem restrição de extensões/MIME.

1. Resumo Executivo

A auditoria revelou um cenário de **extrema criticidade na camada de autorização e isolamento de dados**. A aplicação adota uma arquitetura em que a interface (Flutter Web) assume a responsabilidade exclusiva de controlar o acesso a dados e funcionalidades, enquanto a API REST subjacente (<https://codeflowbr.online:8080/api/v1/>) responde a requisições sem autenticação ou com verificação nula de privilégios.

3

CRÍTICA

4

ALTA

3

MÉDIA

2

BAIXA

4

PONTOS FORTES

DISTRIBUIÇÃO DE VULNERABILIDADES POR SEVERIDADE



ACHADOS POR CATEGORIA AUDITADA

1. Banco Sem Tranca	3
2. Permissão Nav.	2
3. IDOR	4
4. Chaves / Hardcode	2
5. Injeção / XSS	3
Pontos Fortes	4

Vetor de Ataque Mais Crítico Comprovado: O endpoint GET

<https://codeflowbr.online:8080/api/v1/customer> foi testado e retornou com sucesso a lista completa de clientes, contendo **nomes completos, números de CPF, números de cartão de fidelidade, logins, e-mails, saldos de pontos e carteira, endereços completos e o identificador do tenant** de todos os usuários cadastrados, sem exigir nenhum cabeçalho de autorização ou token. Em paralelo, o endpoint PUT <https://codeflowbr.online:8080/api/v1/user> aceita alterações diretas de password e role para qualquer ID, permitindo **Account Takeover global e elevação instantânea para Administrador**.

2. Pontos Fortes e Riscos Centrais

PROTEGIDO

Pontos Fortes Verificados

- **Sanitização em `EditarCustomerCall`:** O método `escapeStringForJson` em `api_calls.dart:1515` protege caracteres `"`, `\` e quebras de linha em edições de perfil.
- **Higienização estrita de CNPJ:** Em `get_company_by_cnpj.dart:15–18`, dígitos são filtrados por regex estrita (`[^0–9]`) e tamanho validado antes de invocar a `BrasilAPI`.
- **Infraestrutura centralizada de Tokens:** A classe `ApiManager` (`api_manager.dart:24–34`) já possui código preparado para anexar `Authorization: Bearer` automaticamente quando `_accessToken` for preenchido.
- **Resiliência de UI com `errorBuilder`:** Em `listagem_de_parceiros_widget.dart:495` e `listagem_de_cupom_widget.dart:670`, o uso de fallback visual evita quebra em cascata na renderização.

RISCO

Vulnerabilidades Críticas Centrais

- **Ausência de RLS / Tenant Middleware:** Listagens e dashboards retornam dados agregados de todos os inquilinos sem isolamento.
- **Autorização 100% no Cliente:** O backend não valida se o usuário possui papel de admin ao receber comandos em rotas de configuração, cupons, produtos e usuários.
- **IDOR em Leitura, Edição e Exclusão:** Endpoints de exclusão (`DELETE /discount/{id}`, `/product/{id}`) e leitura aceitam qualquer ID numérico sem checagem de posse.
- **Injeção de JSON por Interpolação de Strings:** Criação e alteração de entidades montam JSON via interpolação de strings sem serializador.

3. Tabela Detalhada de Achados por Categoria

SEVERIDADE	ARQUIVO & LINHA	CATEGORIA	DESCRIÇÃO E EXPLORABILIDADE
CRÍTICA	lib/backend/api_requests/api_calls.dart:181	1. Banco Sem Tranca	Vazamento irrestrito de clientes (LGPD): ObterUsuariosCall (GET /api/v1/customer) sem filtro de tenant ou usuário. Retorna CPF, nomes, cartões e dados de todos os clientes sem autenticação.
ALTA	lib/backend/api_requests/api_calls.dart:18	1. Banco Sem Tranca	Vazamento de métricas globais no Dashboard: ObterDashboardCompletoCall (GET /api/v1/dashboard) expõe faturamento e indicadores agregados de todos os inquilinos.
ALTA	lib/app_constants.dart:5 api_calls.dart:687, 914, 1021	1. Banco Sem Tranca	Tenant Hopping / Tenant controlado pelo cliente: tenantId = '1' hardcoded e injetado pelo frontend no payload JSON. O servidor aceita qualquer idTenant informado no body.
CRÍTICA	lib/pages/login/login_widget.dart:407 lib/flutter_flow/nav/navigation.dart:326	2. Permissão Nav.	Gate cosmético de papel e rotas abertas: Rotas de admin no GoRouter têm requireAuth = false. O desvio entre Parceiro e Admin ocorre apenas via if (LoginCall.role(...) == 2) local.
CRÍTICA	lib/backend/api_requests/api_calls.dart:283, 925, 1032	2. Permissão Nav.	Elevação de privilégio arbitrária via payload: role é enviado livremente pelo cliente na criação/edição de usuários (AtualizarDadosCall, criarParceiro). Qualquer usuário pode se promover a Admin (role 2).
CRÍTICA	lib/backend/api_requests/api_calls.dart:268-301	3. IDOR	Account Takeover irrestrito de qualquer usuário: AtualizarDadosCall (PUT /api/v1/user) altera senha, e-mail e papel de qualquer usuário passando o id no corpo da requisição.
ALTA	lib/backend/api_requests/api_calls.dart:143-158	3. IDOR	Exclusão concorrente de cupons: ExcluirDescontoCall (DELETE /api/v1/discount/{idDiscount}) permite que qualquer parceiro delete cupons pertencentes a outros parceiros.
ALTA	lib/backend/api_requests/api_calls.dart:1262-1281	3. IDOR	Espionagem de faturamento e métricas: ObterDashboardParceiroCall (GET /api/v1/dashboard/partner/{partnerId}) exibe métricas financeiras e transações de qualquer parceiro pelo ID.

SEVERIDADE	ARQUIVO & LINHA	CATEGORIA	DESCRIÇÃO E EXPLORABILIDADE
ALTA	lib/backend/api_requests/api_calls.dart:532-550	3. IDOR	Acesso a propostas comerciais confidenciais: ObterPropostaPorIdCall (GET /api/v1/proposta/{idProposta}) expõe dados societários e financeiros de propostas de qualquer empresa.
MÉDIA	lib/backend/api_requests/api_calls.dart:20, 696...	4. Chaves / Hardcode	Hardcode de URL de produção e portas expostas: https://codeflowbr.online:8080/api/v1/ repetida em centenas de linhas. Exposição direta da porta 8080 sem WAF/Cloudflare.
BAIXA	lib/auth/custom_auth/custom_auth_manager.dart:102	4. Chaves / Hardcode	Persistência de tokens em LocalStorage (XSS Risk): SharedPreferences no Flutter Web armazena tokens JWT em texto claro no window.localStorage, sem flag HttpOnly.
ALTA	lib/backend/api_requests/api_calls.dart:276, 619, 676, 907...	5. Injeção / XSS	Injeção de JSON por interpolação de strings: Mais de 15 chamadas montam JSON via concatenação '{ "login": "\${login}" }' sem escape, permitindo quebra de sintaxe e injeção de atributos.
MÉDIA	lib/custom_code/actions/upload_photo.dart:20 upload_file.dart:20	5. Injeção / XSS	Upload anônimo e irrestrito de arquivos: POST /api/v1/photo e /arquivo sem autenticação e sem validação client-side de extensão (.php, .html, .svg com XSS).
BAIXA	lib/flutter_flow/flutter_flow_util.dart:198 contrato_parceiro_widget.dart:133	5. Injeção / XSS	Injeção de Esquema de URL em launchURL: launchUrl(Uri.parse(url)) não valida esquemas seguros (http/https), permitindo execução de pseudoprotocolos (ex: javascript:).

4. Recomendações Priorizadas (Plano de Remediação)

Prioridade P1 (Imediata — Próximas 24 horas)

- 1. Fechamento Imediato de Endpoints Abertos:** Exigir autenticação obrigatória via Bearer JWT em **todas** as rotas da API (/customer, /dashboard, /discount, /user, /partner, /photo). Bloquear imediatamente requisições anônimas.
- 2. Bloqueio de Parâmetros Privilegiados:** Remover os campos `role` e `tenant.id` das requisições de criação e edição enviadas pelo cliente. O backend deve associar o usuário ao tenant e atribuir a role com base exclusiva na identidade autenticada no token JWT.
- 3. Correção de IDOR em AtualizarDadosCall:** Garantir que a rota `PUT /api/v1/user` altere exclusivamente os dados do usuário autenticado no token (extraíndo o ID do contexto da sessão no servidor, e não do body).

Prioridade P2 (Curto Prazo — Próximos 7 dias)

- 4. Implementação de Middleware de Tenant e Checagem de Posse:** Criar validação server-side em todas as queries e rotas de exclusão/edição: `WHERE tenant_id = :authenticated_tenant_id AND partner_id = :authenticated_partner_id`.
- 5. Substituição de Interpolação JSON por `jsonEncode()`:** Refatorar as chamadas em `api_calls.dart` para utilizar `Map<String, dynamic>` serializados via `json.encode(bodyMap)`, eliminando 100% dos riscos de JSON Injection e quebra de payload.
- 6. Proteção de Rotas Administrativas no Frontend:** Atualizar o `nav.dart` para definir `requireAuth: true` em rotas privadas e implementar `redirect guards` baseados em token válido.

Prioridade P3 (Médio Prazo — Próximas 2 semanas)

- 7. Variáveis de Ambiente e Gateway Reverso:** Extrair a URL da API para `String.fromEnvironment('API_BASE_URL')` e posicionar o backend atrás de um Reverse Proxy / WAF (Cloudflare/Nginx) com HTTPS na porta padrão 443.
- 8. Validação Estrita de Uploads:** Implementar validação de MIME type e magic bytes no upload de arquivos, além de desabilitar a execução de scripts no diretório estático de fotos.
- 9. Sanitização de Esquemas em `launchURL`:** Validar se o esquema da URI pertence exclusivamente a `https` ou `http` antes de invocar `launchUrl()`.

5. Issues para o GitHub (Markdown Pronto para Cópia)

As issues abaixo foram formatadas em Markdown completo, prontas para inclusão no rastreador de tarefas do repositório.

Issue #1: [Segurança] Vazamento de dados de clientes e falta de isolamento de inquilino em /customer

CRÍTICA

```
--- ISSUE 1 ---
**Título:** [Segurança] Vazamento de dados pessoais de clientes e ausência de filtro de inquilino em /customer
**Labels:** `security`, `severity:critical`, `lgpd`, `tenant-isolation`

### Descrição do Problema
O endpoint de listagem de clientes responde a requisições sem exigir autenticação ou validação de inquilino, retornando registros de múltiplos clientes cadastrados na base com dados extremamente sensíveis.

### Evidência de Código
- **Arquivo:** `lib/backend/api_requests/api_calls.dart` (Linhas 180-205)
  ```dart
 class ObterUsuariosCall {
 static Future call() async {
 return ApiManager.instance.makeApiCall(
 callName: 'obterUsuarios',
 apiUrl: 'https://codeflowbr.online:8080/api/v1/customer',
 callType: ApiCallType.GET,
 headers: {},
 params: {},
);
 }
 }
  ```

### Prova de Conceito (Explorabilidade)
Uma simples requisição HTTP não autenticada realizada contra o servidor retorna o payload com centenas de clientes:
  ```bash
 curl -i "https://codeflowbr.online:8080/api/v1/customer"
  ```

### Campos expostos:
  `cpf`, `cardNumber`, `name`, `login` (e-mail), `adress` (rua, número, bairro, cidade, CEP), `wallet` (saldo de pontos).

### Impacto
Violação gravíssima da Lei Geral de Proteção de Dados (LGPD) e vazamento de base de clientes de todos os inquilinos da plataforma para qualquer usuário externo.

### Sugestão de Correção
1. Adicionar middleware de autenticação obrigatória JWT na rota `/api/v1/customer`.
2. No backend, obter o `tenant_id` exclusivamente das claims do JWT validado.
3. Filtrar as consultas com `WHERE tenant_id = :tenant_id`. Se o solicitante for parceiro, restringir a clientes vinculados ou proibir a listagem global.

### Critérios de Aceite
- [ ] Requisições não autenticadas em `GET /api/v1/customer` retornam status `401 Unauthorized`.
- [ ] Usuários autenticados só conseguem visualizar clientes pertencentes ao seu próprio inquilino (`tenant_id`).
- [ ] O cabeçalho `Authorization: Bearer` é incluído em `ObterUsuariosCall` em `api_calls.dart`.
--- FIM ISSUE 1 ---
```

Issue #2: [Segurança] IDOR e Elevação de Privilégio Arbitrária via PUT /api/v1/user**CRÍTICA**

--- ISSUE 2 ---

****Título:**** [Segurança] IDOR e Account Takeover com elevação de papel em PUT /api/v1/user****Labels:**** `security`, `severity:critical`, `idor`, `broken-access-control`**### Descrição do Problema**

O método `AtualizarDadosCall` permite o envio de um corpo JSON contendo o `id` do usuário a ser alterado, bem como sua nova senha (`password`) e papel administrativo (`role`). O backend atualiza o usuário sem verificar se o solicitante possui permissão para editar aquele ID específico ou se possui papel de administrador para alterar a role.

Evidência de Código- ****Arquivo:**** `lib/backend/api\_requests/api\_calls.dart` (Linhas 268-300)

```

`dart
class AtualizarDadosCall {
  static Future call({
    String? id = '',
    String? email = '',
    String? password = '',
    String? inviteCode = '',
    int? role,
  }) async {
    final ffApiRequestBody = '''
{
  "id": "${id}",
  "isActive": true,
  "inviteCode": "${inviteCode}",
  "login": "${email}",
  "password": "${password}",
  "role": ${role}
}''';
  ...

```

Impacto

- ****Account Takeover Total:**** Um atacante pode alterar a senha de qualquer usuário ou administrador conhecendo apenas seu ID numérico sequencial.
- ****Elevação de Privilégios Imediata:**** Ao submeter `"role": 2`, qualquer conta é promovida a Administrador global.

Sugestão de Correção

1. Desabilitar a passagem de `id` e `role` pelo corpo da requisição em rotas de autoatendimento.
2. Identificar o usuário pelo token de sessão JWT (`sub` claim) no backend.
3. Proibir que o próprio usuário edite o campo `role` da conta. Alterações de papel devem exigir endpoint específico e privilégio de superadmin.

Critérios de Aceite

- [] O usuário não pode alterar dados de contas de terceiros passando IDs arbitrários.
- [] O atributo `role` não é alterável via endpoint de atualização de perfil.
- [] Qualquer tentativa de alteração sem token válido resulta em `401 Unauthorized`.

--- FIM ISSUE 2 ---

Issue #3: [Segurança] IDOR na Exclusão e Consulta de Recursos (Cupons, Produtos, Dashboards)

ALTA

--- ISSUE 3 ---

****Título:**** [Segurança] IDOR na exclusão e visualização de cupons, produtos e dashboards
****Labels:**** `security`, `severity:high`, `idor`, `authorization`

Descrição do Problema

Várias operações críticas recebem identificadores numéricos diretamente na URL e realizam mutações ou exibição de dados sem validar se o recurso pertence ao usuário ou parceiro autenticado.

Evidência de Código

```
- **Arquivo:** `lib/backend/api_requests/api_calls.dart`
- Linha 143: `ExcluirDescontoCall`: `DELETE /api/v1/discount/${idDiscount}`
- Linha 309: `DeletarProdutoCall`: `DELETE /api/v1/product/${idProduto}`
- Linha 1269: `ObterDashboardParceiroCall`: `GET /api/v1/dashboard/partner/${partnerId}`
- Linha 1290: `ObterCuponsDoParceiroCall`: `GET /api/v1/discount/partner/user/${partnerId}`
```

```
```dart
```

```
class ExcluirDescontoCall {
 static Future call({String? idDiscount = ''}) async {
 return ApiManager.instance.makeApiCall(
 callName: 'excluirDesconto',
 apiUrl: 'https://codeflowbr.online:8080/api/v1/discount/${idDiscount}',
 callType: ApiCallType.DELETE,
);
 }
}
```

### ### Impacto

- Parceiros comerciais podem excluir cupons e produtos de estabelecimentos concorrentes.
- Espionagem comercial de receitas e métricas ao consultar o dashboard de outros parceiros alterando o `partnerId`.

### ### Sugestão de Correção

No backend, validar a propriedade de cada recurso antes de efetuar exclusão ou retorno:

```
```sql
DELETE FROM discount WHERE id = :idDiscount AND partner_id = :authenticated_partner_id;
```
```

Se nenhuma linha for afetada ou se o parceiro for diferente, responder com `403 Forbidden`.

### ### Critérios de Aceite

- [ ] Tentar deletar um cupom de outro parceiro retorna `403 Forbidden`.
- [ ] Tentar acessar o dashboard com um `partnerId` divergente do token autenticado retorna `403 Forbidden`.

--- FIM ISSUE 3 ---

## Issue #4: [Segurança] Injeção de JSON por Interpolação de Strings em Requisições de API

ALTA

--- ISSUE 4 ---

**\*\*Título:\*\*** [Segurança] Injeção de JSON por interpolação direta de strings em api\_calls.dart  
**\*\*Labels:\*\*** `security`, `severity:high`, `injection`, `api-client`

### ### Descrição do Problema

Diversas funções construtoras de requisições HTTP em `api\_calls.dart` criam strings JSON usando interpolação direta (``'{ "nome": "\${nome}" }'``) em vez de serializar mapas via `jsonEncode()`. Caracteres como aspas (```) e quebras de linha inseridos pelo usuário em campos de texto corrompem o payload ou permitem a injeção de parâmetros adicionais.

### ### Evidência de Código

- **\*\*Arquivo:\*\*** `lib/backend/api\_requests/api\_calls.dart` (Linhas 676-693, 907-954, 1098-1147, etc.)  
 ```dart

```
final ffApiRequestBody = '''
{
  "description": "${descricao}",
  "discount": "${porcentagem}",
  "isActive": true,
  "rules": "${rules}"
}''';
```

(Apenas em `EditarCustomerCall` na linha 1515 houve adição pontual de `escapeStringForJson`).

Impacto

- Falha e quebra da aplicação quando usuários inserem caracteres especiais ou descrições multilinha.
- Injeção de atributos não autorizados no JSON (ex.: injetar `"role": 2, "dummy": "` em campos de texto).

Sugestão de Correção

Refatorar todas as chamadas para utilizar mapas do Dart serializados com `json.encode`:

```
```dart
final Map payload = {
 'description': descricao,
 'discount': porcentagem,
 'isActive': true,
 'rules': rules,
};
final body = json.encode(payload);
```
```

Critérios de Aceite

- [] Nenhuma chamada em `api\_calls.dart` utiliza interpolação de strings em bloco (``'{ ... }'``) para montar JSON.
 - [] Textos com aspas e caracteres de controle são enviados e persistidos corretamente sem erro 400 ou injeção.
- FIM ISSUE 4 ---

Issue #5: [Segurança] Ausência de Validação de Esquema e Vetor de Redirecionamento em launchURL

BAIXA

--- ISSUE 5 ---

****Título:**** [Segurança] Ausência de validação de protocolo seguro em launchURL (XSS / Open Redirect)
****Labels:**** `security`, `severity:low`, `xss`, `client-security`

Descrição do Problema

O método auxiliar `launchURL` recebe strings de URL de contratos e banners e dispara diretamente `launchUrl(uri)` sem validar se o protocolo pertence a uma lista permitida (`https` ou `http`). No Flutter Web, protocolos como `javascript:` podem ser acionados no navegador do cliente se alimentados por fontes manipuladas no banco.

Evidência de Código

- ****Arquivo:**** `lib/flutter\_flow/flutter\_flow\_util.dart` (Linhas 198-205)

```
dart
Future launchURL(String url) async {
  var uri = Uri.parse(url);
  try {
    await launchUrl(uri);
  } catch (e) {
    throw 'Could not launch $uri: $e';
  }
}
```

- ****Arquivo:**** `lib/pages/contrato\_parceiro/contrato\_parceiro\_widget.dart` (Linha 133):

```
dart
await launchURL(getJsonField(FFAppState().parceiro, r'''$.partner.contract''').toString());
```

Impacto

Possibilidade de execução de scripts locais ou redirecionamento de usuários administrativos para páginas maliciosas de phishing a partir de links de contratos controlados por atacantes.

Sugestão de Correção

Aplicar verificação estrita de protocolo:

```
dart
Future launchURL(String url) async {
  final uri = Uri.tryParse(url.trim());
  if (uri == null || (!uri.isScheme('http') && !uri.isScheme('https'))) {
    throw 'Protocolo de URL inválido ou inseguro';
  }
  await launchUrl(uri, mode: LaunchMode.externalApplication);
}
```

Critérios de Aceite

- [] URLs com prefixo `javascript:`, `data:` ou desconhecidas são bloqueadas com exceção controlada.
- [] Apenas links iniciando em `http://` ou `https://` são abertos pelo aplicativo.

--- FIM ISSUE 5 ---